High-Speed Graph Coloring on GPUs

Ghadeer Alabandi, Evan Powers, and Martin Burtscher*

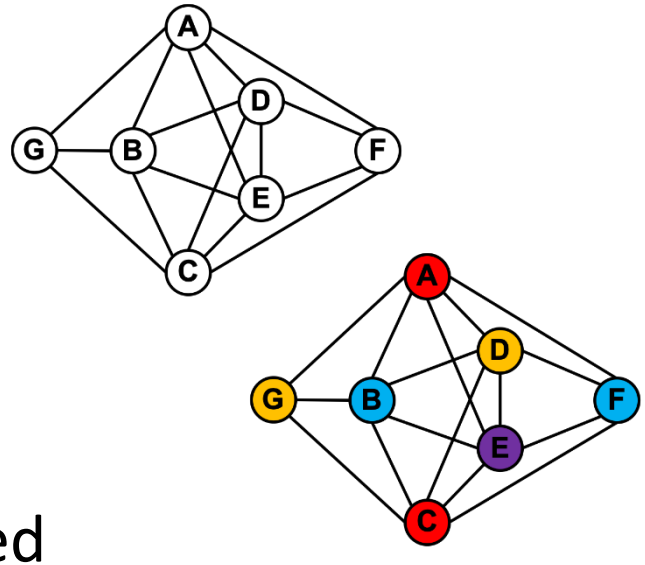
GPU Technology Conference (GTC) 2021





Graph Coloring

- The coloring of an undirected graph G = (V, E) is a mapping C from vertices to colors such that
  - C(i) ≠ C(j) for every edge (i, j) ∈ E
  - Total number of colors is small
- Serial algorithm is trivial
  - Select an uncolored vertex
  - Color it with “best” possible color
  - Repeat until entire graph is colored



First color Second color Third color Fourth color Fifth color Sixth color

Highlights

- Our parallel ECL-GC algorithm
  - Increases parallelism by breaking dependencies
    - Call these techniques "shortcuts"
  - Produces deterministic color assignment
    - Serial, parallel w/o shortcuts, parallel with shortcuts
- CUDA code
  - Demonstrates efficient implementation of shortcuts
- Results
  - 2.5 times more intrinsic parallelism (1.0x - 12.4x)
  - 2.9 and 4.0 times faster than fastest GPU & CPU codes

Background

- Finding a graph coloring with the minimum number of colors is NP-hard
- Several heuristics exist to color graphs
  - Guarantee that adjacent vertices have distinct colors
  - Require different numbers of colors
- These heuristics provide different tradeoffs
  - Trade off coloring quality and execution time

Prior GC Algorithms

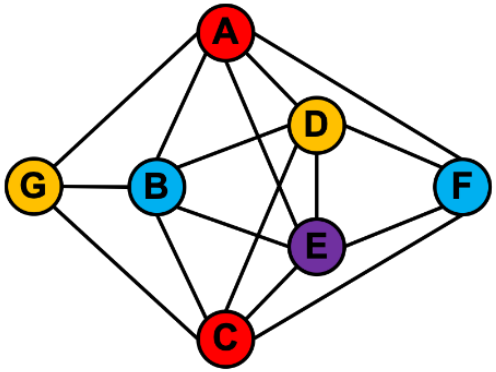
Greedy Algorithm

- Assign a random priority to each vertex
- Select the highest-priority uncolored vertex
- Color this vertex with the best possible color
  - The colors are ordered (see below)
    - First color is the most preferred color
- Repeat steps 2 and 3 until entire graph is colored

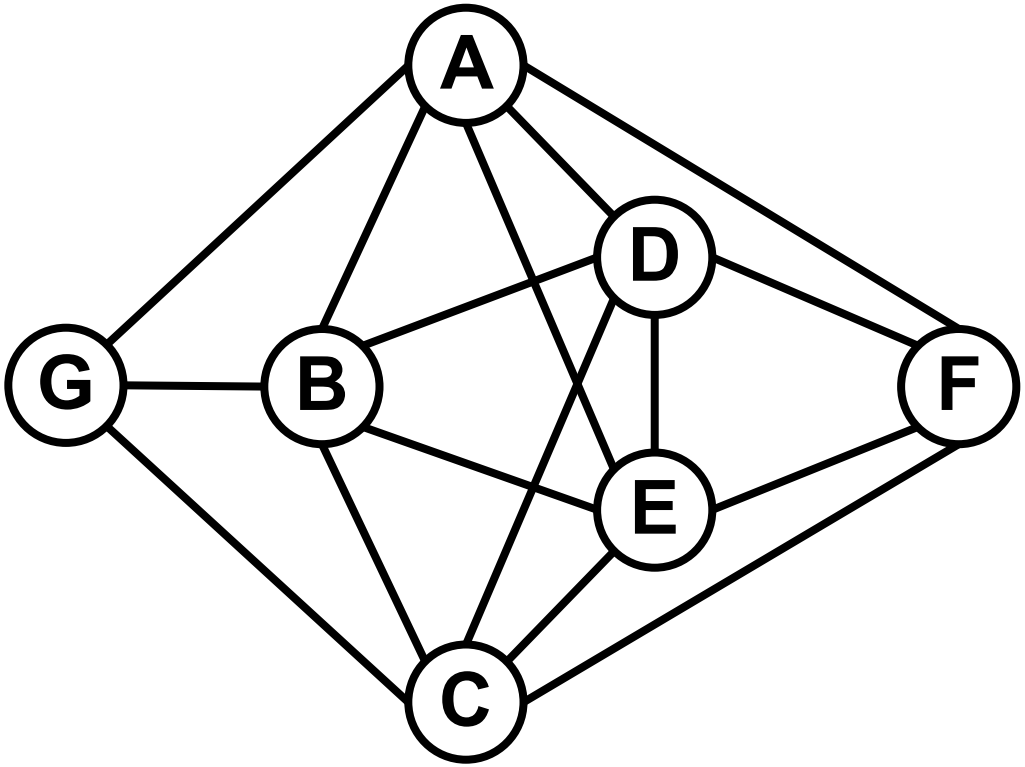


Priority Heuristics

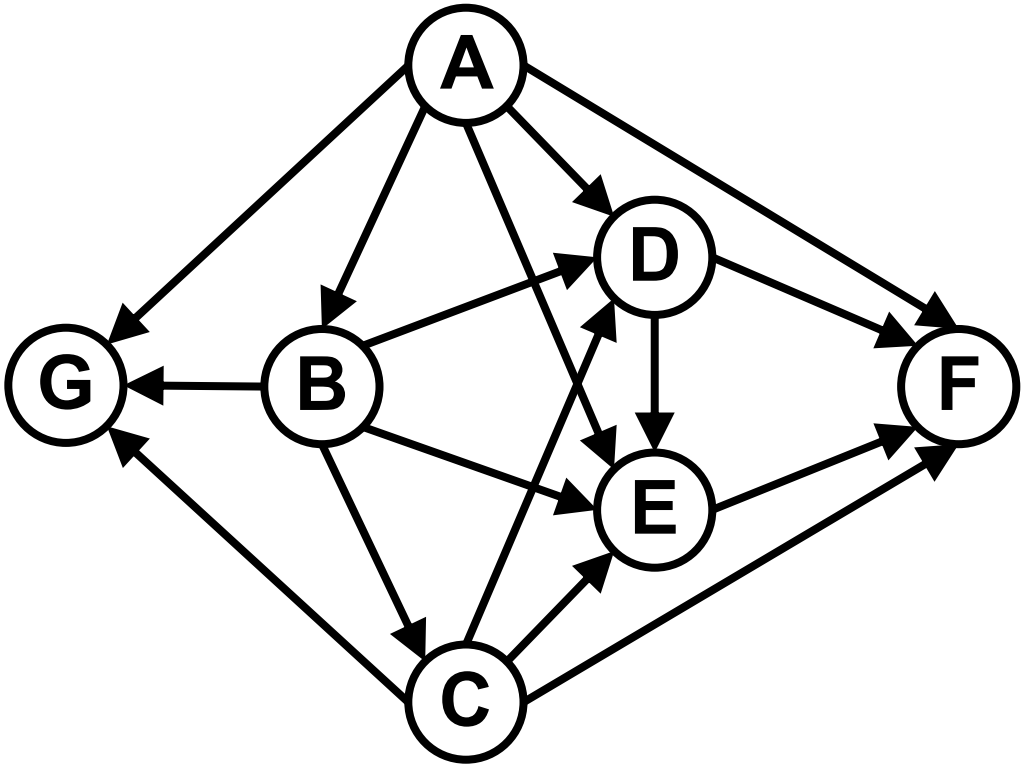
- First fit (FF)
- Random (R)
- Largest degree first (LDF)
- Smallest degree last (SDL)
- Saturation degree (SD)
- Incidence degree (ID)



LDF Example: Input Graph

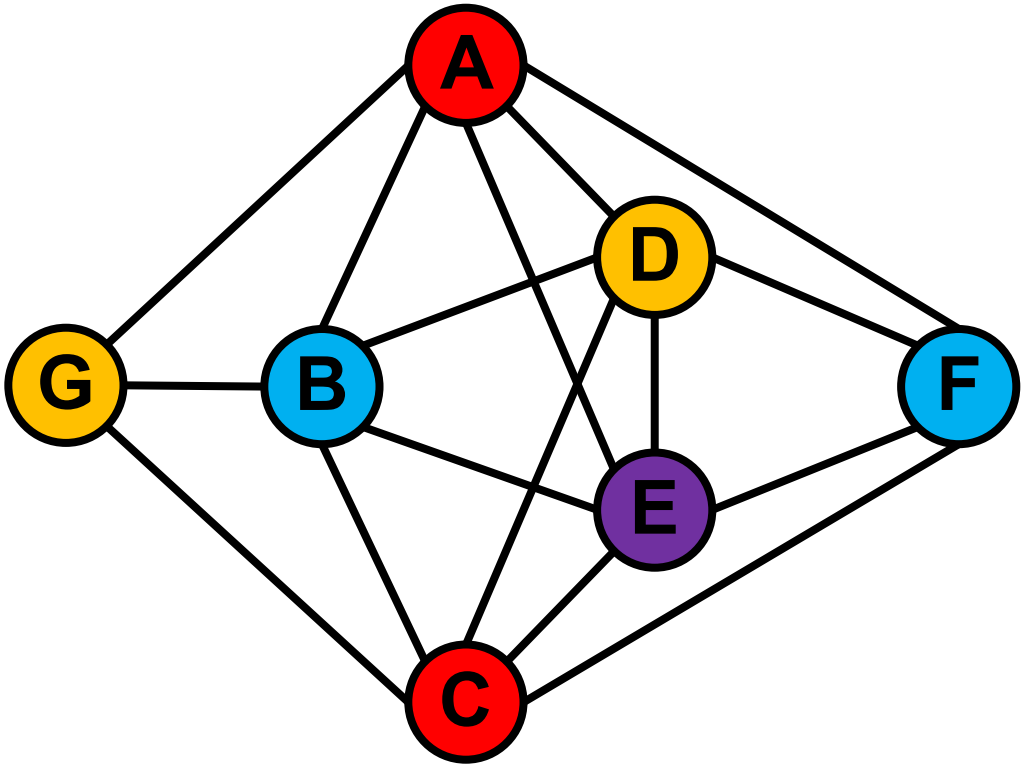


LDF Example: Imposed Priority DAG



From higher- to lower-degree vertex (alpha tie breaker)

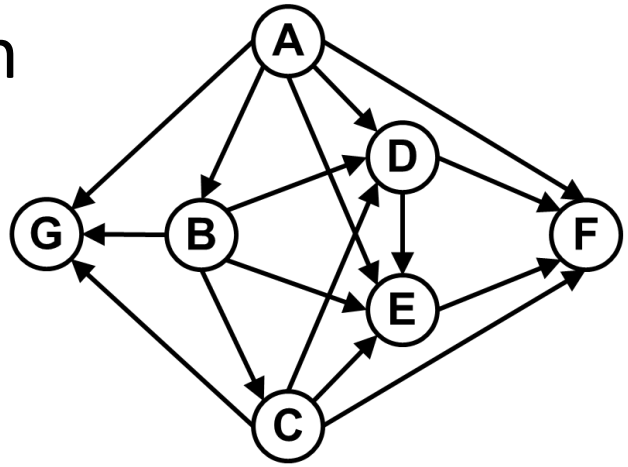
LDF Example: Resulting Coloring



First color Second color Third color Fourth color Fifth color Sixth color

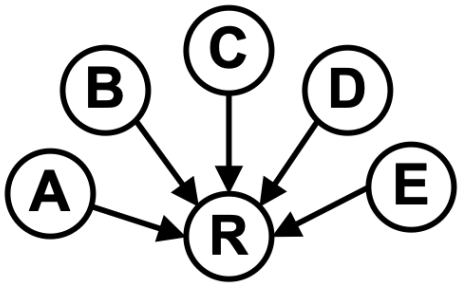
Jones-Plassmann Parallel Algorithm

- Many parallel graph coloring algorithms follow JP approach
- Initialization step
  - Compute direction of edges
    - Based on a priority heuristic
- Parallel computation steps
  - Check every uncolored vertex
  - Color ready vertices with best possible color
  - Repeat until entire graph has been colored



Possible Colors

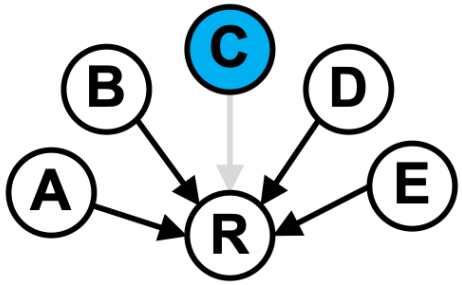
- In JP, each vertex has list of k+1 possible colors
  - k is number of higher-priority neighbors
  - When neighbor is colored, that color is deleted from list
  - Vertex is ultimately colored with best remaining color
- JP often uses bitmaps for implementing these lists
  - Each bit represents one color
    - "1" means the color is still possible
    - "0" means the color is not available



First color Second color Third color Fourth color Fifth color Sixth color

Possible Colors

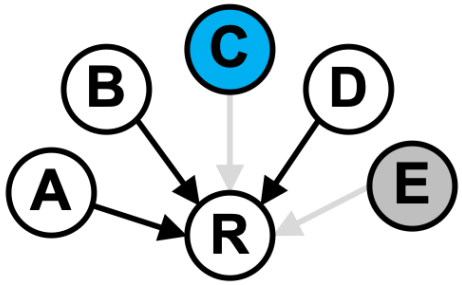
- In JP, each vertex has list of *k*+1 possible colors
  - *k* is number of higher-priority neighbors
  - When neighbor is colored, that color is deleted from list
  - Vertex is ultimately colored with best remaining color
- JP often uses bitmaps for implementing these lists
  - Each bit represents one color
    - "1" means the color is still possible
    - "0" means the color is not available



First color Second color Third color Fourth color Fifth color Sixth color

Possible Colors

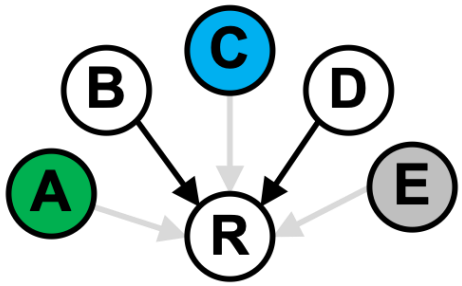
- In JP, each vertex has list of *k*+1 possible colors
  - *k* is number of higher-priority neighbors
  - When neighbor is colored, that color is deleted from list
  - Vertex is ultimately colored with best remaining color
- JP often uses bitmaps for implementing these lists
  - Each bit represents one color
    - "1" means the color is still possible
    - "0" means the color is not available



First color Second color Third color Fourth color Fifth color Sixth color

Possible Colors

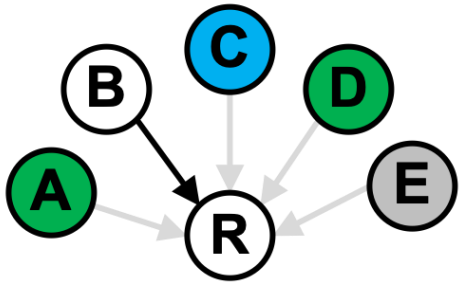
- In JP, each vertex has list of *k*+1 possible colors
  - *k* is number of higher-priority neighbors
  - When neighbor is colored, that color is deleted from list
  - Vertex is ultimately colored with best remaining color
- JP often uses bitmaps for implementing these lists
  - Each bit represents one color
    - "1" means the color is still possible
    - "0" means the color is not available



First color Second color Third color Fourth color Fifth color Sixth color

Possible Colors

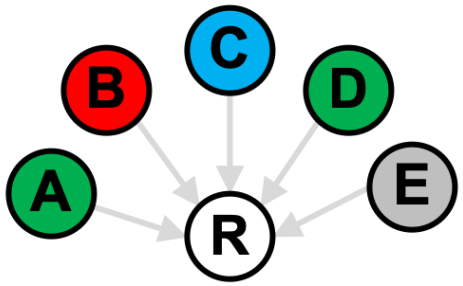
- In JP, each vertex has list of *k*+1 possible colors
  - *k* is number of higher-priority neighbors
  - When neighbor is colored, that color is deleted from list
  - Vertex is ultimately colored with best remaining color
- JP often uses bitmaps for implementing these lists
  - Each bit represents one color
    - "1" means the color is still possible
    - "0" means the color is not available



First color Second color Third color Fourth color Fifth color Sixth color

Possible Colors

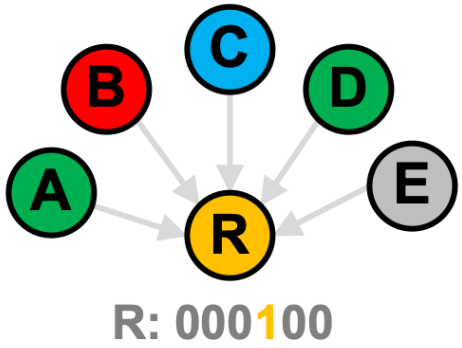
- In JP, each vertex has list of *k*+1 possible colors
  - *k* is number of higher-priority neighbors
  - When neighbor is colored, that color is deleted from list
  - Vertex is ultimately colored with best remaining color
- JP often uses bitmaps for implementing these lists
  - Each bit represents one color
    - "1" means the color is still possible
    - "0" means the color is not available



First color Second color Third color Fourth color Fifth color Sixth color

Possible Colors

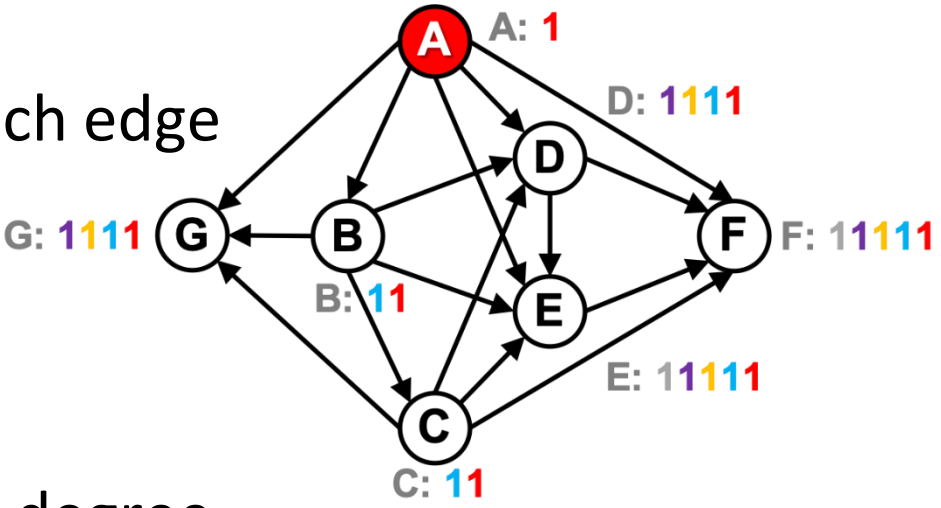
- In JP, each vertex has list of k+1 possible colors
  - k is number of higher-priority neighbors
  - When neighbor is colored, that color is deleted from list
  - Vertex is ultimately colored with best remaining color
- JP often uses bitmaps for implementing these lists
  - Each bit represents one color
    - "1" means the color is still possible
    - "0" means the color is not available



First color Second color Third color Fourth color Fifth color Sixth color

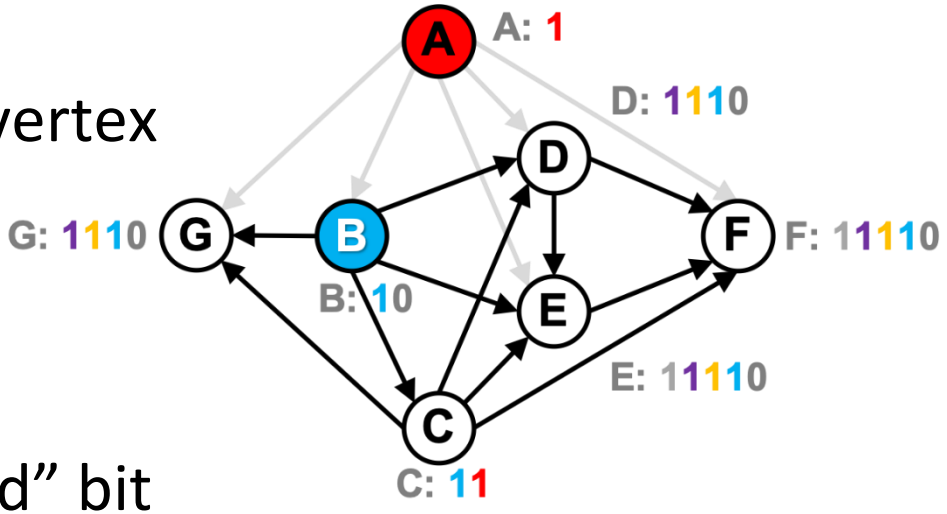
Jones-Plassmann Algorithm

- Initialization step
  - Compute direction of each edge
    - Based on LDF heuristic
  - Initialize bitmaps
- Operation
  - Go from higher to lower degree
  - Use alphabetic ordering to break ties
- Result
  - Turns graph into DAG; already colors vertex A "**red**"



Jones-Plassmann Example

- Computation step 1
  - Check every uncolored vertex
  - Color ready vertices

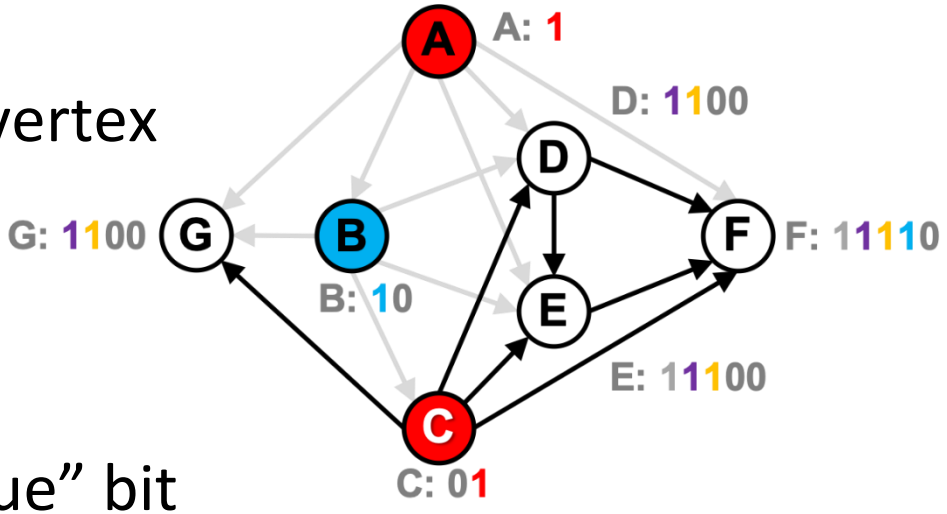


- Operation
  - Neighbors of A clear “red” bit
  - Vertex B is ready
  - Color vertex B “**blue**”

First color Second color Third color Fourth color Fifth color Sixth color

Jones-Plassmann Example

- Computation step 2
  - Check every uncolored vertex
  - Color ready vertices

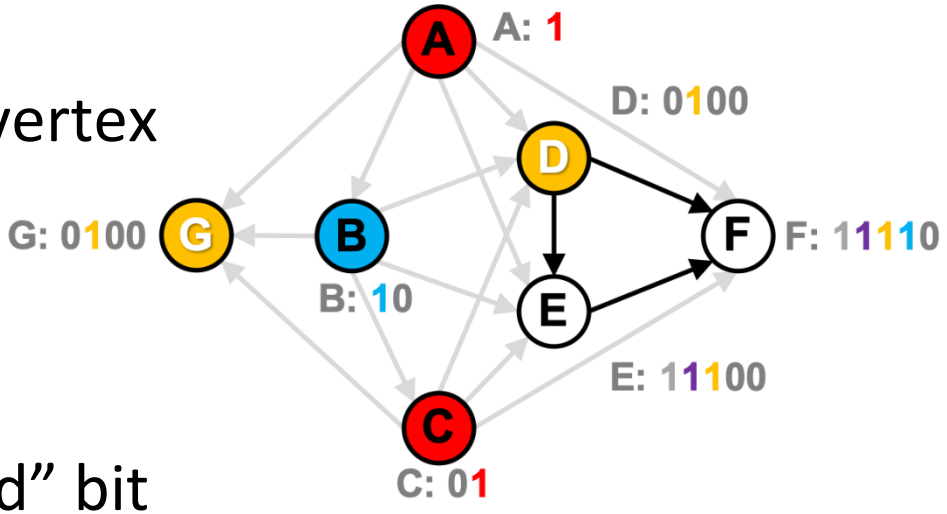


- Operation
  - Neighbors of B clear “blue” bit
  - Vertex C is ready
  - Color vertex C “**red**”

First color Second color Third color Fourth color Fifth color Sixth color

Jones-Plassmann Example

- Computation step 3
  - Check every uncolored vertex
  - Color ready vertices

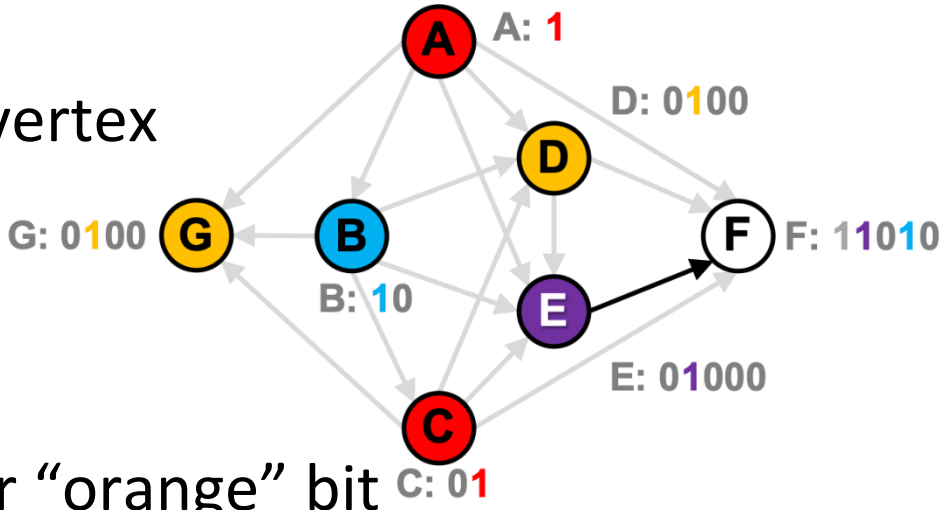


- Operation
  - Neighbors of C clear “red” bit
  - Vertices D & G are ready
  - Color vertices D & G “orange”

First color Second color Third color Fourth color Fifth color Sixth color

Jones-Plassmann Example

- Computation step 4
  - Check every uncolored vertex
  - Color ready vertices

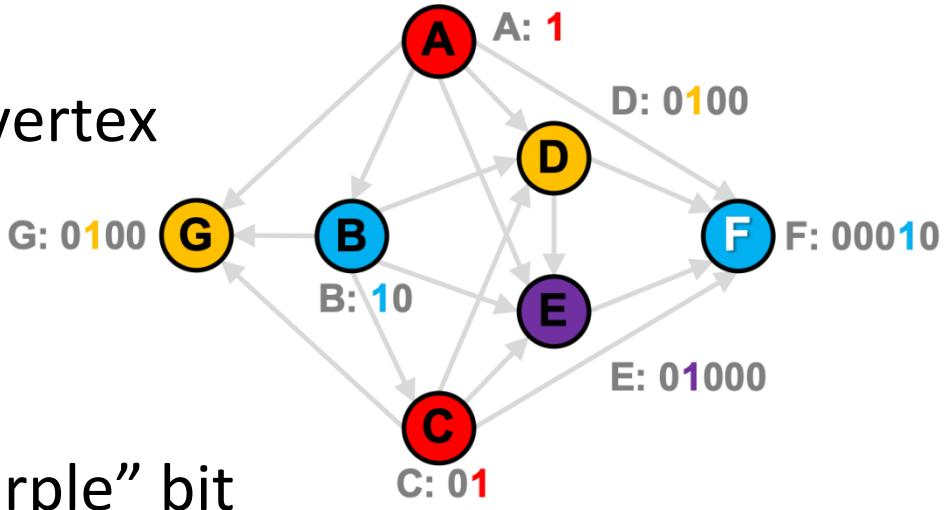


- Operation
  - Neighbors of D & G clear “orange” bit
  - Vertex E is ready
  - Color vertex E “**purple**”

First color Second color Third color Fourth color Fifth color Sixth color

Jones-Plassmann Example

- Computation step 5
  - Check every uncolored vertex
  - Color ready vertices



- Operation
  - Neighbors of E clear “purple” bit
  - Vertex F is ready
  - Color vertex F “**blue**”

JP terminates after 5 steps b/c longest dependence chain has 5 edges

First color Second color Third color Fourth color Fifth color Sixth color

Our ECL-GC Algorithm with Shortcuts

ECL-GC

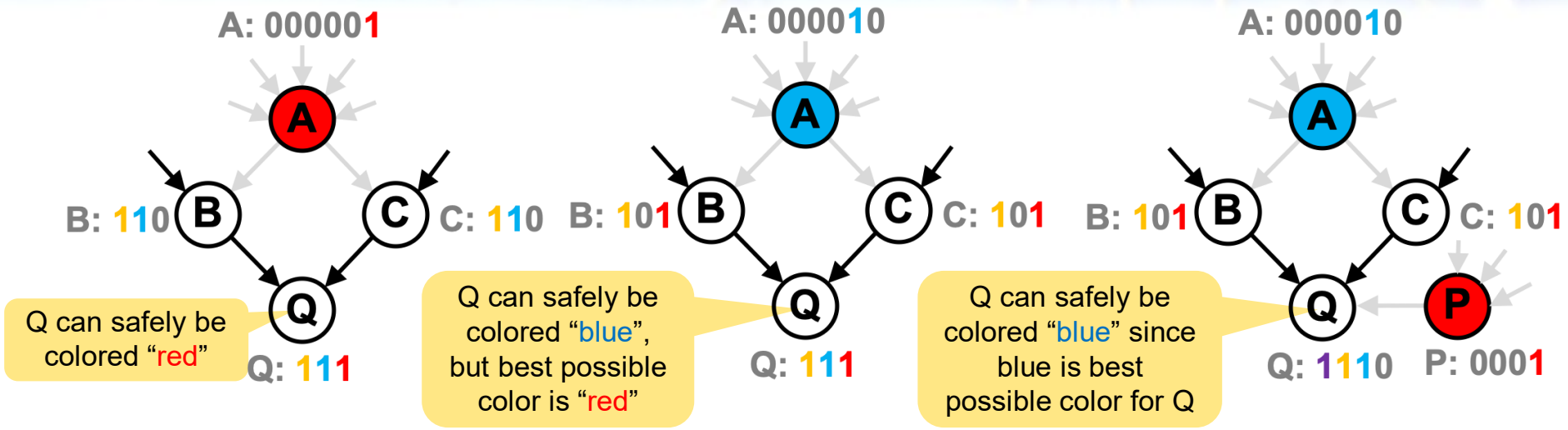
- Our algorithm extends JP-LDF
  - JP-LDF tends to be fast and produces good colorings
- JP-LDF colors vertices with higher degrees first
  - Lower-degree vertices must wait → low parallelism
- JP tends to exhibit limited parallelism
  - Additional non-speculative parallelism may exist!

ECL-GC (cont.)

- ECL-GC exploits intermediate bitmap states
  - Based on systematic search of operations between adjacent vertices' bitmaps
  - Found two cases that yield exploitable info (shortcuts)
    - See paper for derivation

- Our shortcuts minimize the waiting
  - Break dependences to increase parallelism
  - Guarantee same final coloring as JP-LDF

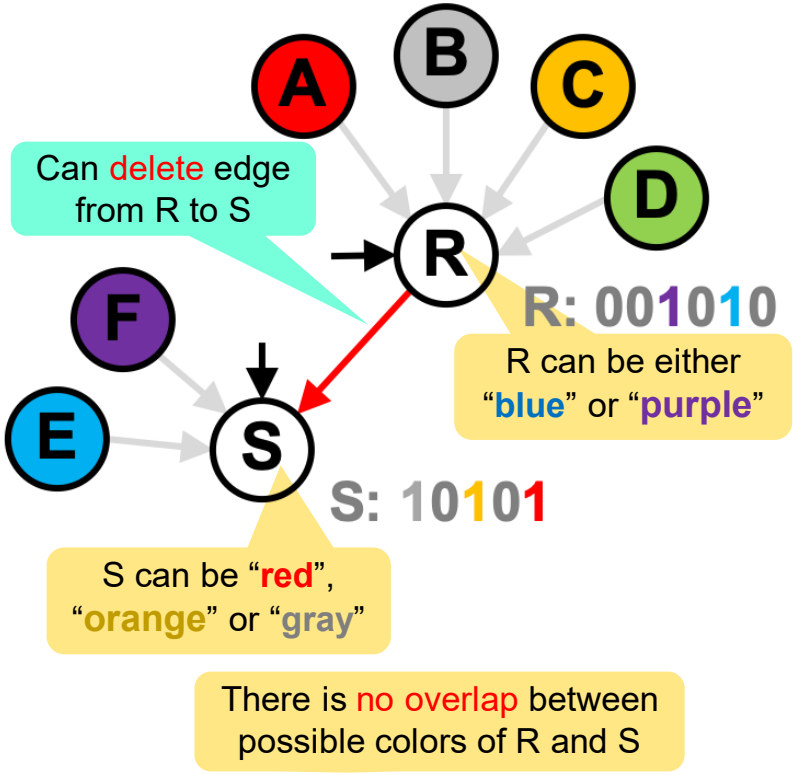
Shortcut 1



- A vertex can be colored with its best possible color
  - As soon as all uncolored higher-priority neighbors are no longer considering that color

First color Second color Third color Fourth color Fifth color Sixth color

Shortcut 2



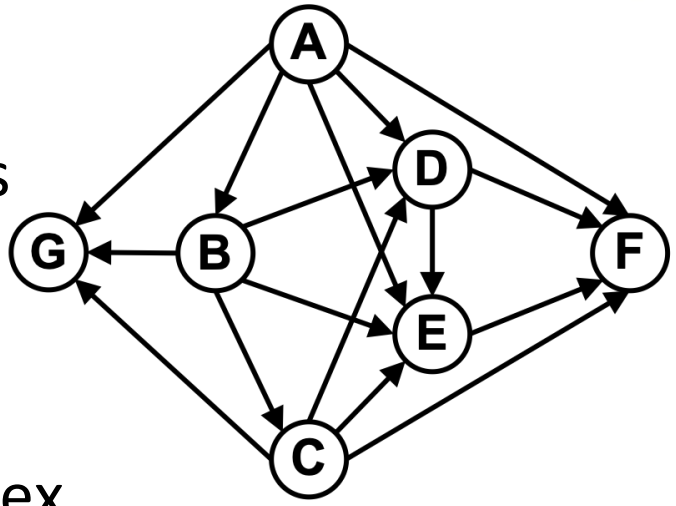
- An edge can be removed
  - As soon as there is no overlap in possible colors
- Benefits
  1. Removes one possible color from bitmap
  2. Speeds up later processing

First color Second color Third color Fourth color Fifth color Sixth color

ECL-GC Operation

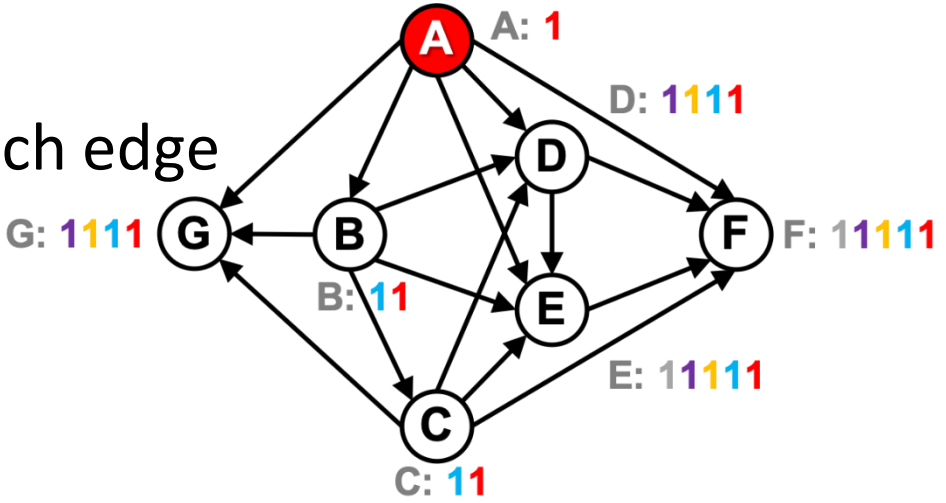
ECL-GC Algorithm

- Initialization step
  - Compute direction of edges
    - Based on priority heuristic
- Computation steps
  - Check every uncolored vertex
  - Color ready vertices with best possible color
  - Apply shortcuts to remaining vertices if feasible
  - Repeat until entire graph has been colored



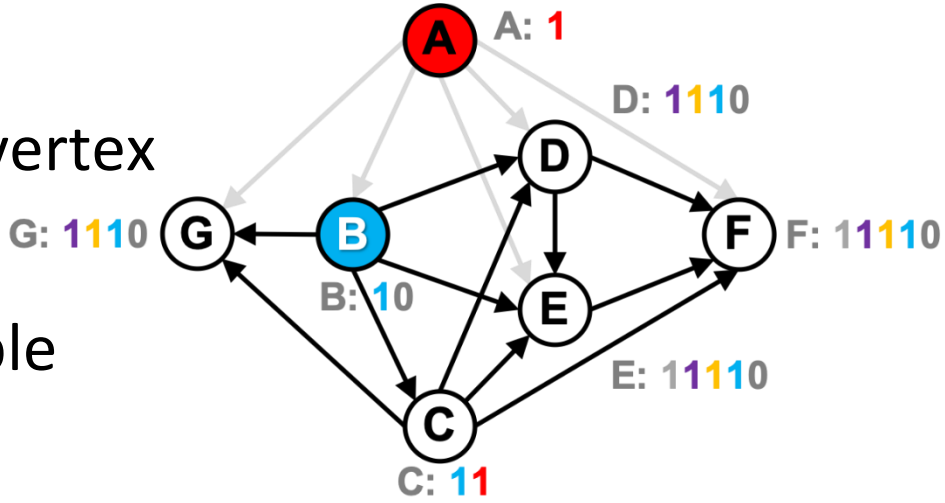
ECL-GC Algorithm

- Initialization step
  - Compute direction of each edge
    - Based on LDF heuristic
  - Initialize bitmaps
- Operation
  - Go from higher to lower degree
  - Use alphabetic ordering to break ties
- Result
  - Turns graph into DAG; already colors vertex A "**red**"



ECL-GC Algorithm

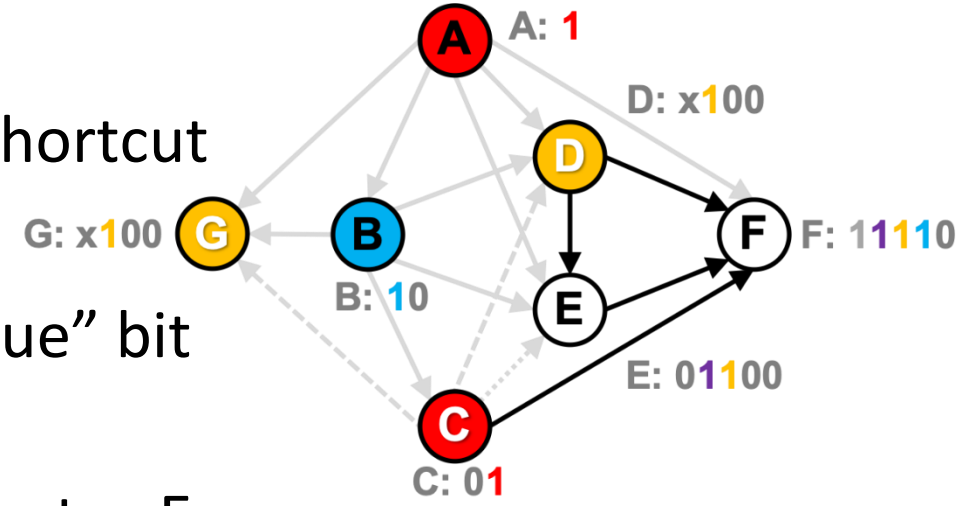
- Computation step 1
  - Check every uncolored vertex
  - Color ready vertices
  - Apply shortcuts if possible
- Operation
  - Neighbors of A clear “red” bit
  - Vertex B is ready
  - Color vertex B “**blue**”



First color Second color Third color Fourth color Fifth color Sixth color

ECL-GC Algorithm

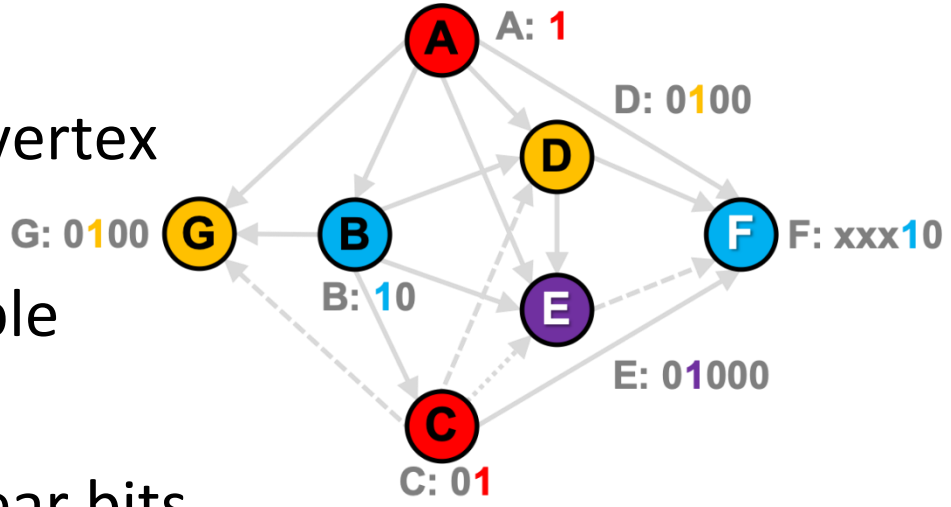
- Computation step 2
  - Color vertices and use shortcut
- Operation
  - Neighbors of B clear “blue” bit
  - Colors vertex C “red”
  - Enables Shortcut 2 on vertex E
    - E can delete edge CE (whether it sees C as 11 or 01)
  - Enables Shortcut 1 on vertices D and G
    - Both can be colored “orange” (whether they see C as 11 or 01)



First color Second color Third color Fourth color Fifth color Sixth color

ECL-GC Algorithm

- Computation step 3
  - Check every uncolored vertex
  - Color ready vertices
  - Apply shortcuts if possible
- Operation
  - Neighbors of C and D clear bits
  - Colors vertex E "purple"
  - Enables Shortcut 1 on vertex F
    - Can be colored "blue" (whether it sees E as 01100 or 01000)



ECL-GC terminates after 3 steps instead of 5

First color Second color Third color Fourth color Fifth color Sixth color

Efficient Implementation

Efficient Shortcut Implementation

- Fully implement shortcuts on low-degree vertices
  - Degree < 32, bitmap fits in 32-bit integer
- Approximate shortcuts on high-degree vertices
  - To avoid slow processing of long bitmaps
  - Only use two integers for shortcut computations
    - First integer specifies best possible color
    - Second integer specifies worst possible color
  - Like JP, still maintains full bitmap for color assignment

Results

Methodology: 2 Devices and 11 Codes

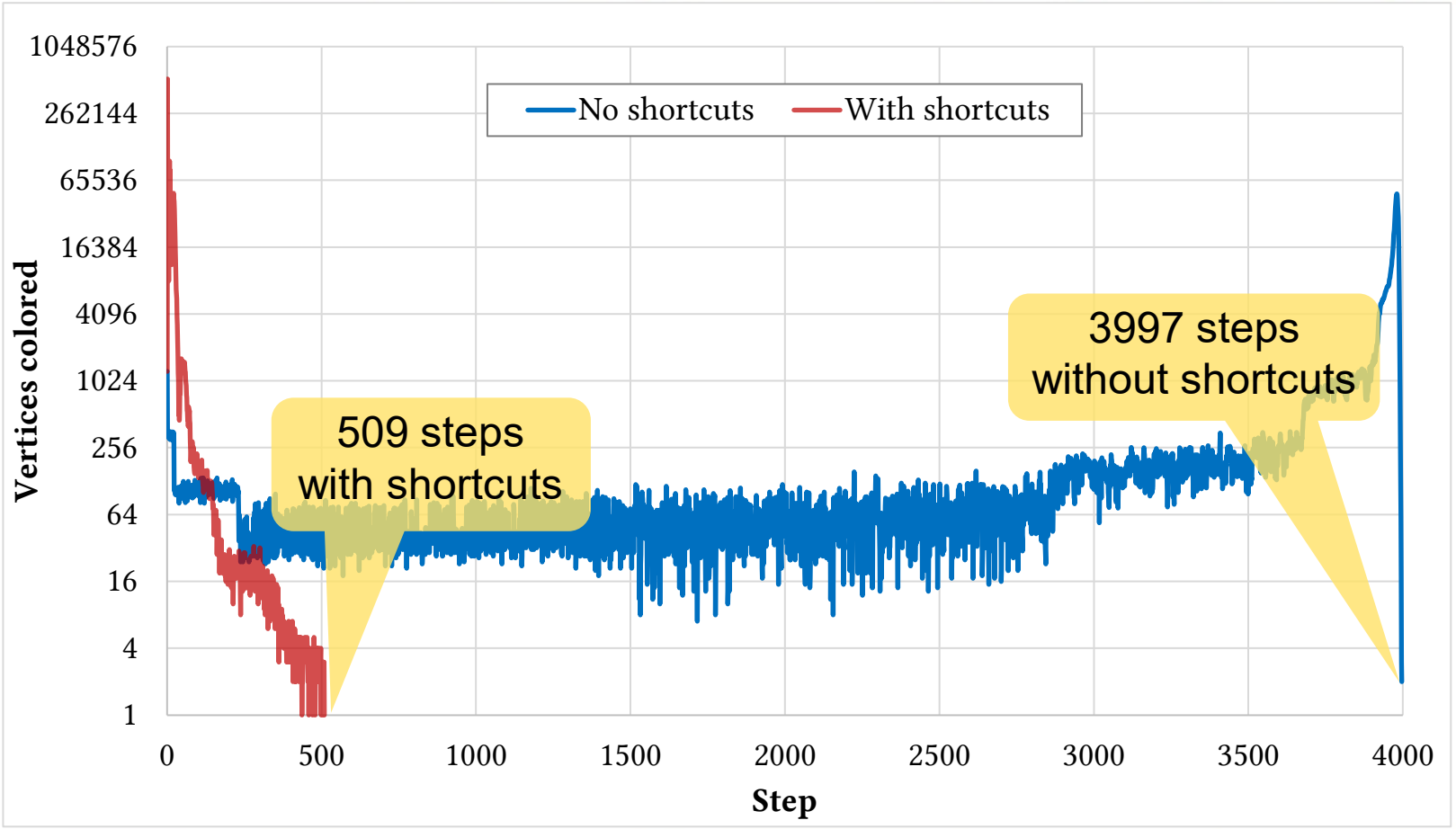
- 1.2 GHz Titan V GPU
  - 5120 cores
  - 4.5 MB L2, 12 GB (652 GB/s)
- 3.1 GHz dual Xeon CPU
  - 20 cores, 40 threads
  - 25 MB L3, 128 GB (68 GB/s)
- One more device in paper
- Comparison
  - 5 GPU codes
  - 3 parallel CPU codes
  - 3 serial CPU codes

Device	Ser/Par	Name	Version
GPU	Parallel	ECL-GC (our code)	1.0
		CUSP	0.5.1
		csrcolor	9.2.88
		Data-wlc	1.0
		Data-pq	1.0
CPU	Parallel	GMMP-NT	
		FirstFit	1.0
		Grappolo	
CPU	Serial	JP-D1	
		FirstFit	1.0
		Boost	1.66.0

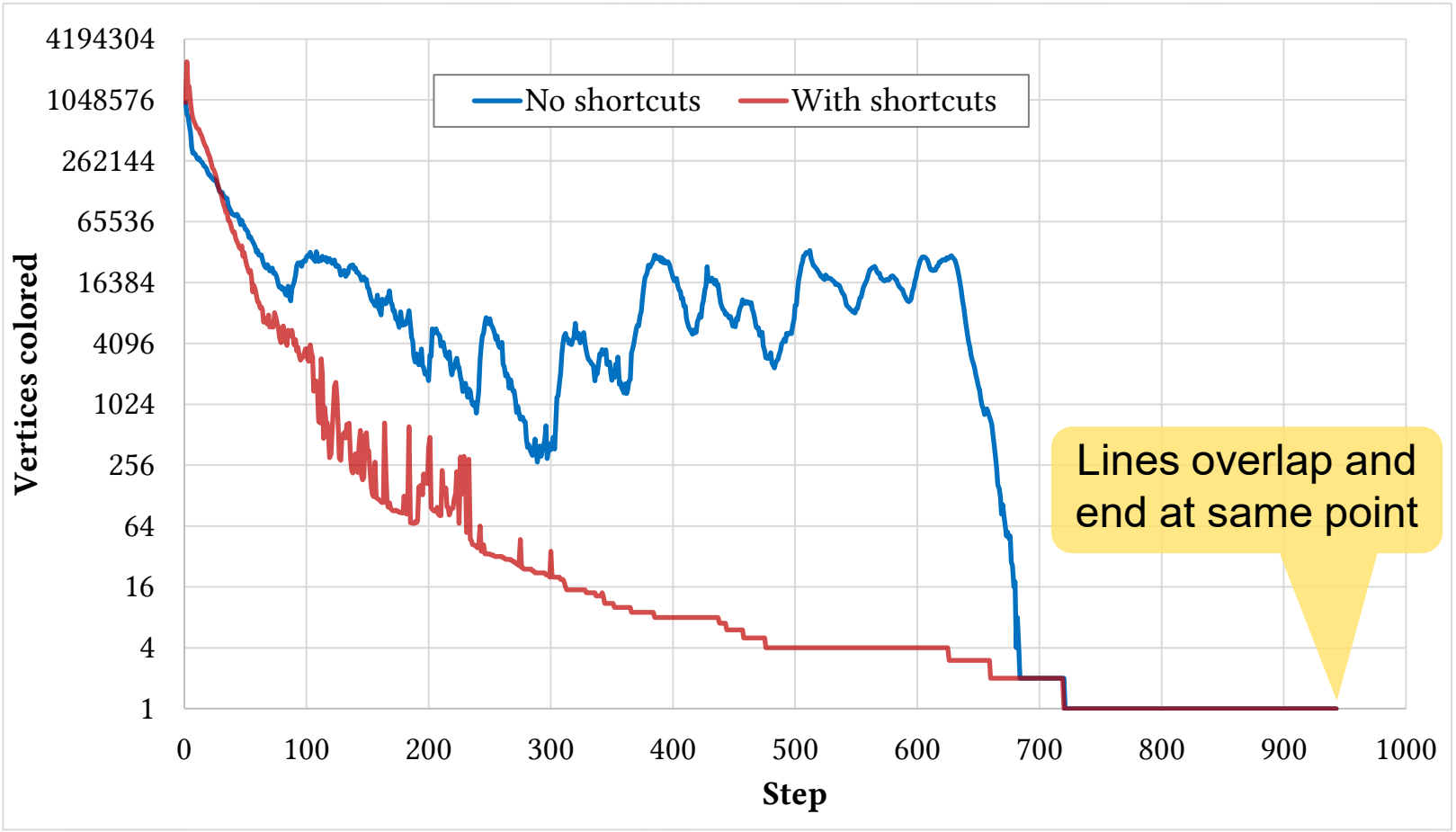
Methodology: 18 Input Graphs

	Graph name	Type	Origin	Vertices	Edges	d_{avg}	d_{max}	d≥32
1	2d-2e20.sym	grid	Galois	1,048,576	4,190,208	4.0	4	0.0%
2	amazon0601	co-purchases	SNAP	403,394	4,886,816	12.1	2,752	3.3%
3	as-skitter	Internet topo.	SNAP	1,696,415	22,190,596	13.1	35,455	6.3%
4	citationCiteseer	publication	SMC	268,495	2,313,294	8.6	1,318	3.6%
5	cit-Patents	patent cites	SMC	3,774,768	33,037,894	8.8	793	3.0%
6	coPapersDBLP	publication	SMC	540,486	30,491,458	56.4	3,299	52.5%
7	delaunay_n24	triangulation	SMC	16,777,216	100,663,202	6.0	26	0.0%
8	europe_osm	road map	SMC	50,912,018	108,109,320	2.1	13	0.0%
9	in-2004	web links	SMC	1,382,908	27,182,946	19.7	21,869	8.4%
10	internet	Internet topo.	SMC	124,651	387,240	3.1	151	0.3%
11	kron_g500-logn21	Kronecker	SMC	2,097,152	182,081,864	86.8	213,904	19.3%
12	r4-2e23.sym	random	Galois	8,388,608	67,108,846	8.0	26	0.0%
13	rmat16.sym	RMAT	Galois	65,536	967,866	14.8	569	11.4%
14	rmat22.sym	RMAT	Galois	4,194,304	65,660,814	15.7	3,687	12.4%
15	soc-LiveJournal1	community	SNAP	4,847,571	85,702,474	17.7	20,333	14.0%
16	uk-2002	web links	SMC	18,520,486	523,574,516	28.3	194,955	18.6%
17	USA-road-d.NY	road map	Dimacs	264,346	730,100	2.8	8	0.0%
18	USA-road-d.USA	road map	Dimacs	23,947,347	57,708,624	2.4	9	0.0%

Amount of Parallelism on Kron Graph



Amount of Parallelism on UK Graph



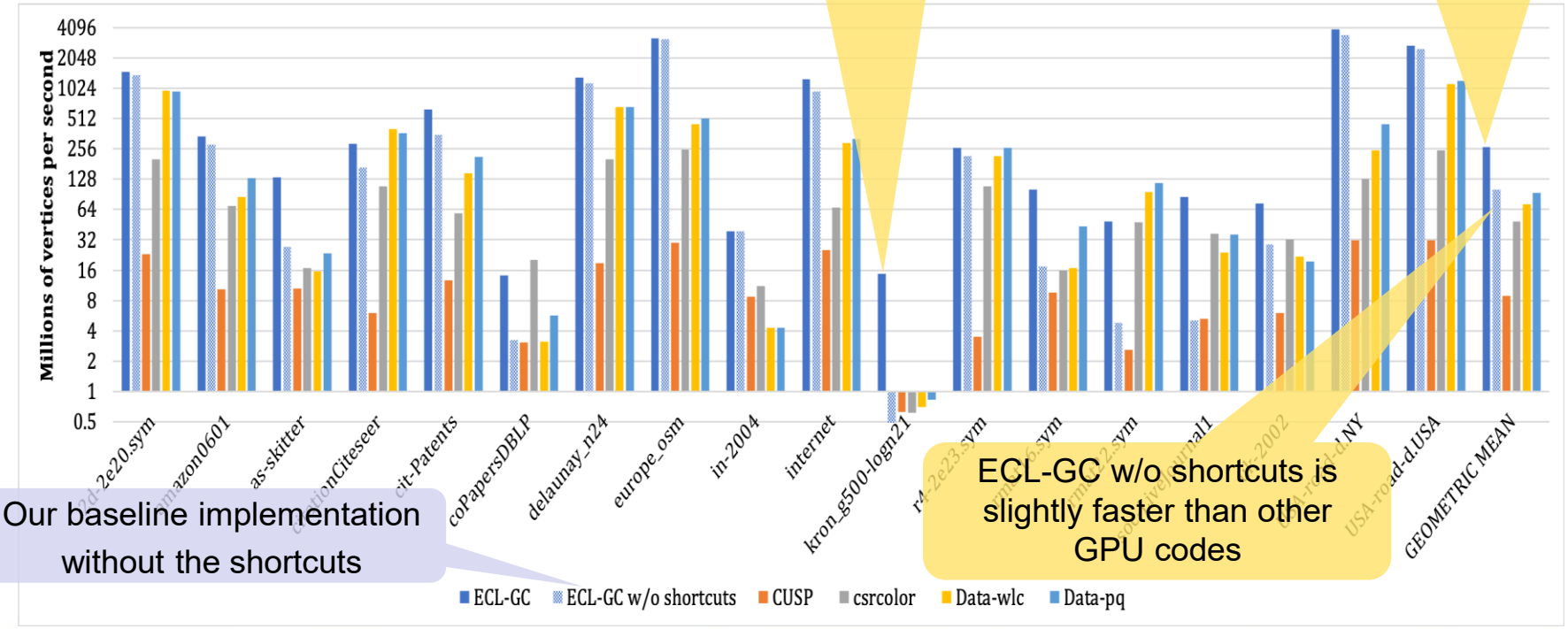
Available Parallelism

Graph	Steps w/o shortcuts	Steps with shortcuts	Avg parallelism w/o shortcuts	Avg parallelism with shortcuts	Increase in parallelism
2d-2e20.sym	14	12	74,898.3	87,381.3	1.17
amazon0601	55	24	7,334.4	16,808.1	2.29
as-skitter	481	73	3,526.9	23,238.6	6.59
citationCiteseer	67	20	4,007.4	13,424.8	3.35
cit-Patents	140	26	26,962.6	145,183.4	5.38
coPapersDBLP	802	338	673.9	1,599.1	2.37
delaunay_n24	25	17	671,088.6	986,895.1	1.47
europe_osm	13	11	3,916,309.1	4,628,365.3	1.18
in-2004	501	490	2,760.3	2,822.3	1.02
internet	27	13	4,616.7	9,588.5	2.08
kron_g500-logn21	3,997	509	524.7	4,120.1	7.85
r4-2e23.sym	30	17	279,620.3	493,447.5	1.76
rmat16.sym	188	30	348.6	2,184.5	6.27
rmat22.sym	644	52	6,512.9	80,659.7	12.38
soc-LiveJournal1	1,095	322	4,427.0	15,054.6	3.40
uk-2002	943	943	19,640.0	19,640.0	1.00
USA-road-d.NY	12	10	22,028.8	26,434.6	1.20
USA-road-d.USA	14	13	1,710,524.8	1,842,103.6	1.08
GEOMEAN	121.0	48.7	16,078.3	39,970.7	2.49

Throughput on Titan V (GPU Codes)

On kron_g500-logn21, ECL-GC is 17 times faster than Data-pq

On average, ECL-GC is 2.9 times faster than the fastest GPU code Data-pq



Our baseline implementation without the shortcuts

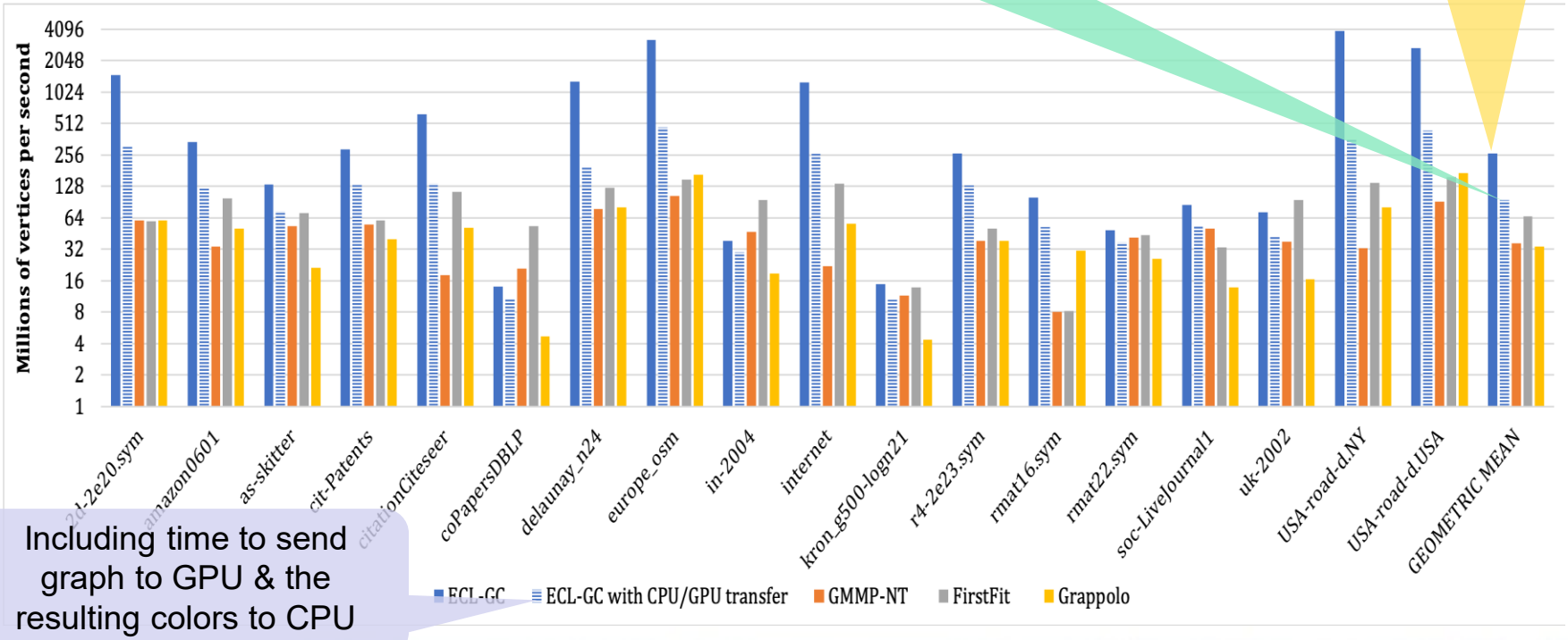
ECL-GC w/o shortcuts is slightly faster than other GPU codes

Throughput (Parallel CPU Codes)

ECL-GC is faster than GMMP-NT & Grappolo on all tested graphs and faster than FirstFit on 15 graphs

Lowers throughput by 2.8x but still faster than CPU codes

On average, ECL-GC is 4.0 times faster than FirstFit

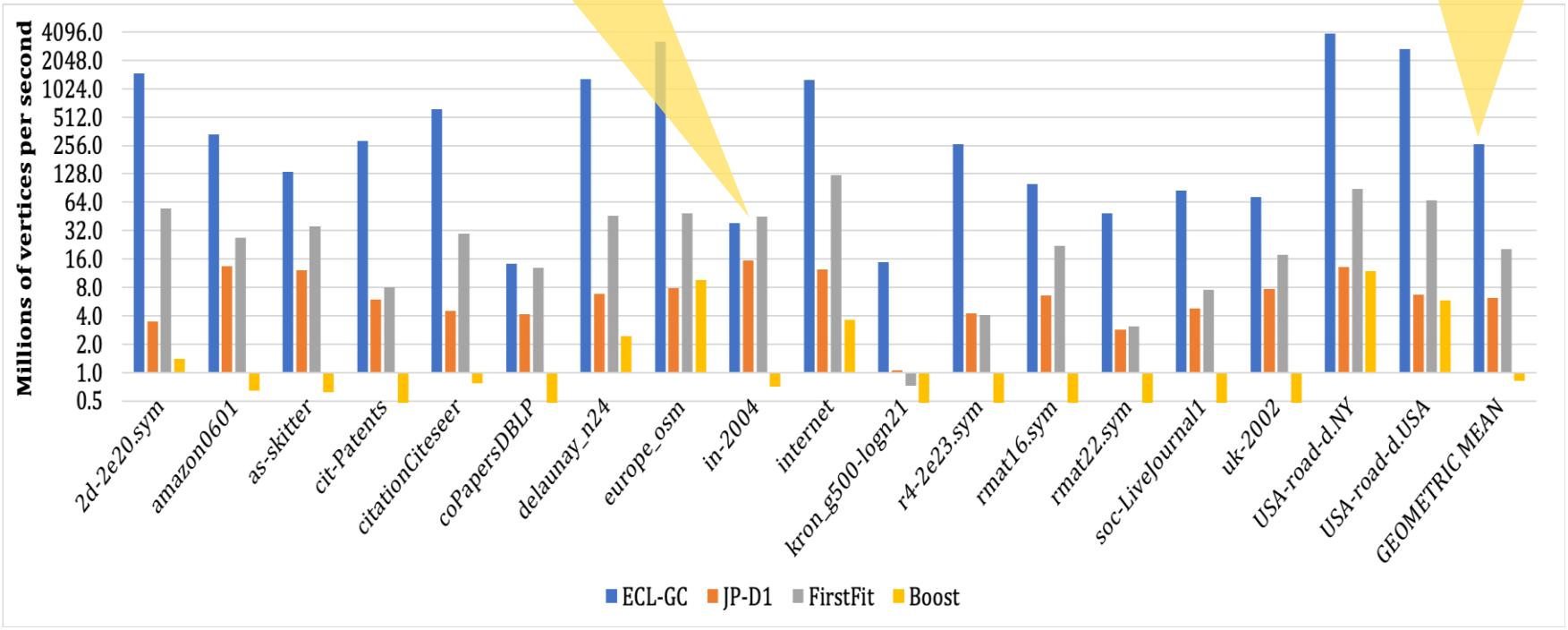


Including time to send graph to GPU & the resulting colors to CPU

Throughput (Serial CPU Codes)

ECL-GC is faster on all tested graphs except in-2004

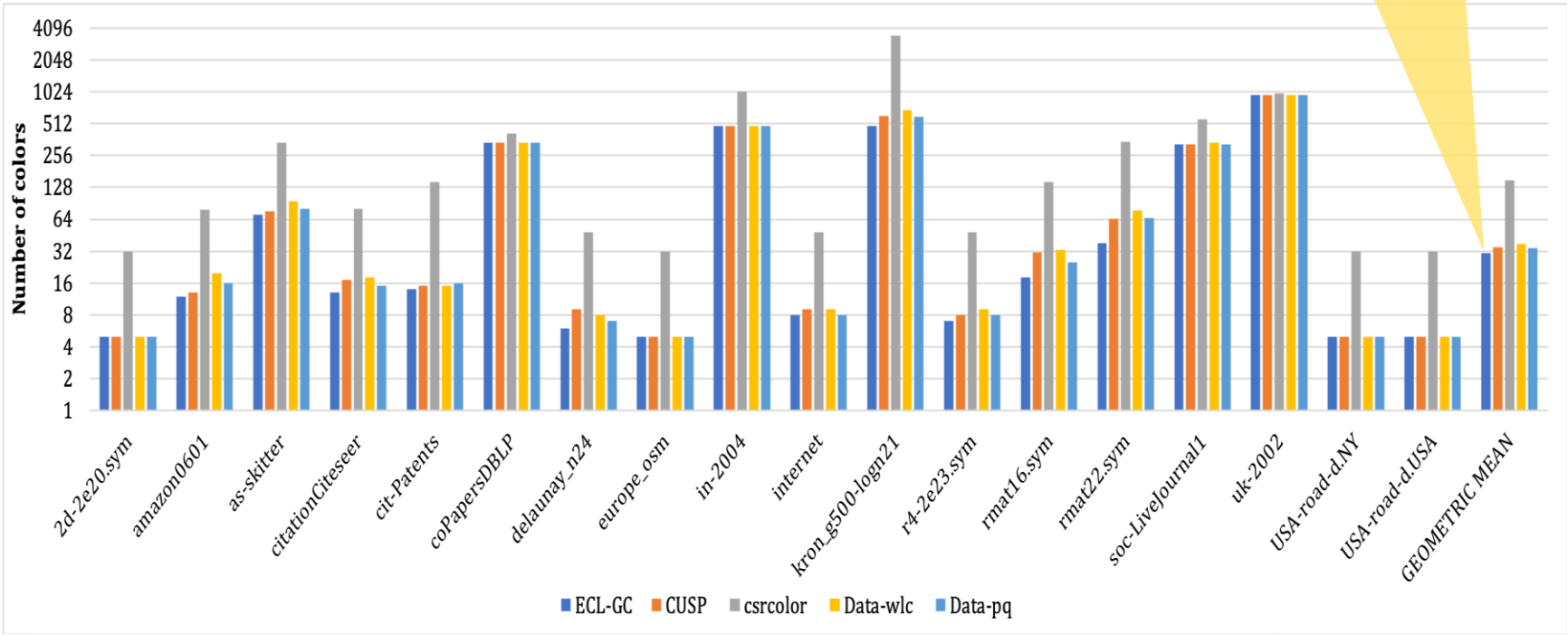
On average, ECL-GC is 13.2 times faster than FirstFit



Number of Colors (GPU Codes)

ECL-GC assigns the smallest or the same number of colors on all tested inputs

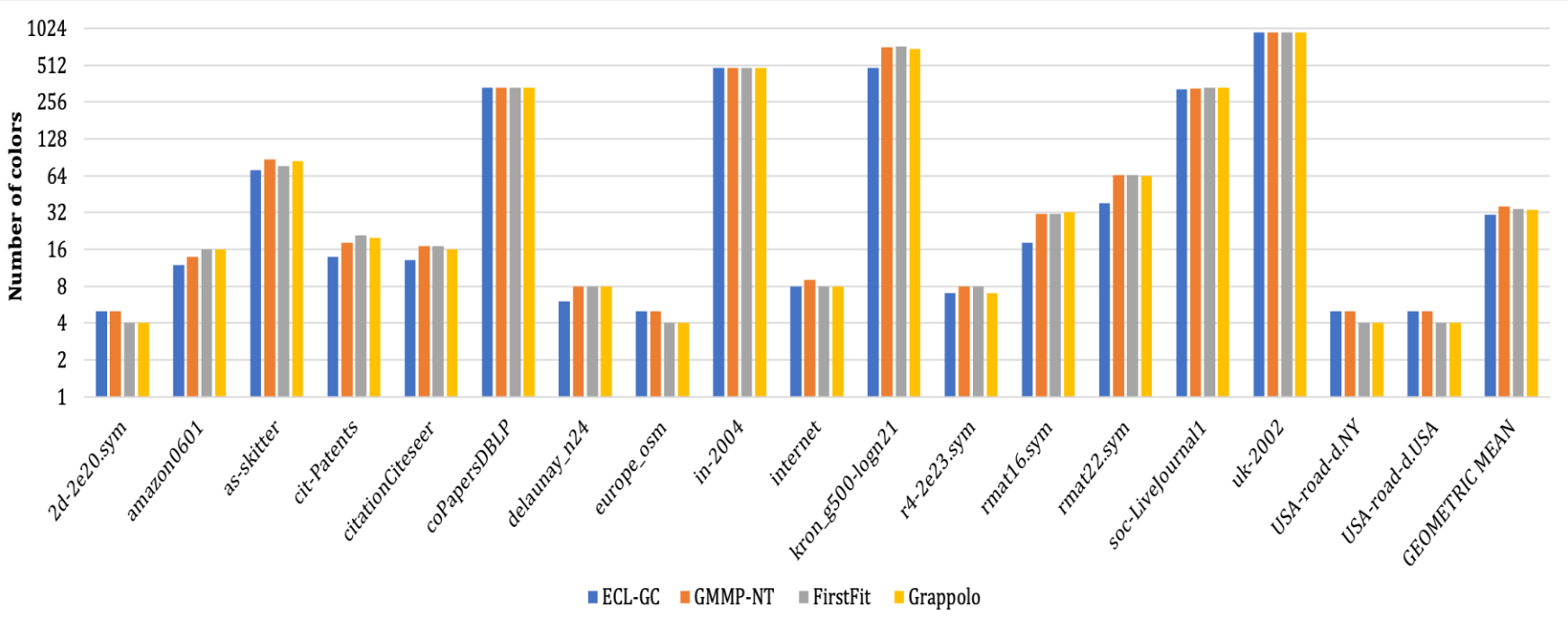
On average, ECL-GC has the best coloring quality



Number of Colors (Parallel CPU Codes)

ECL-GC assign the smallest or the same number of colors as FirstFit & Garppolo on 11 of the 18 graphs

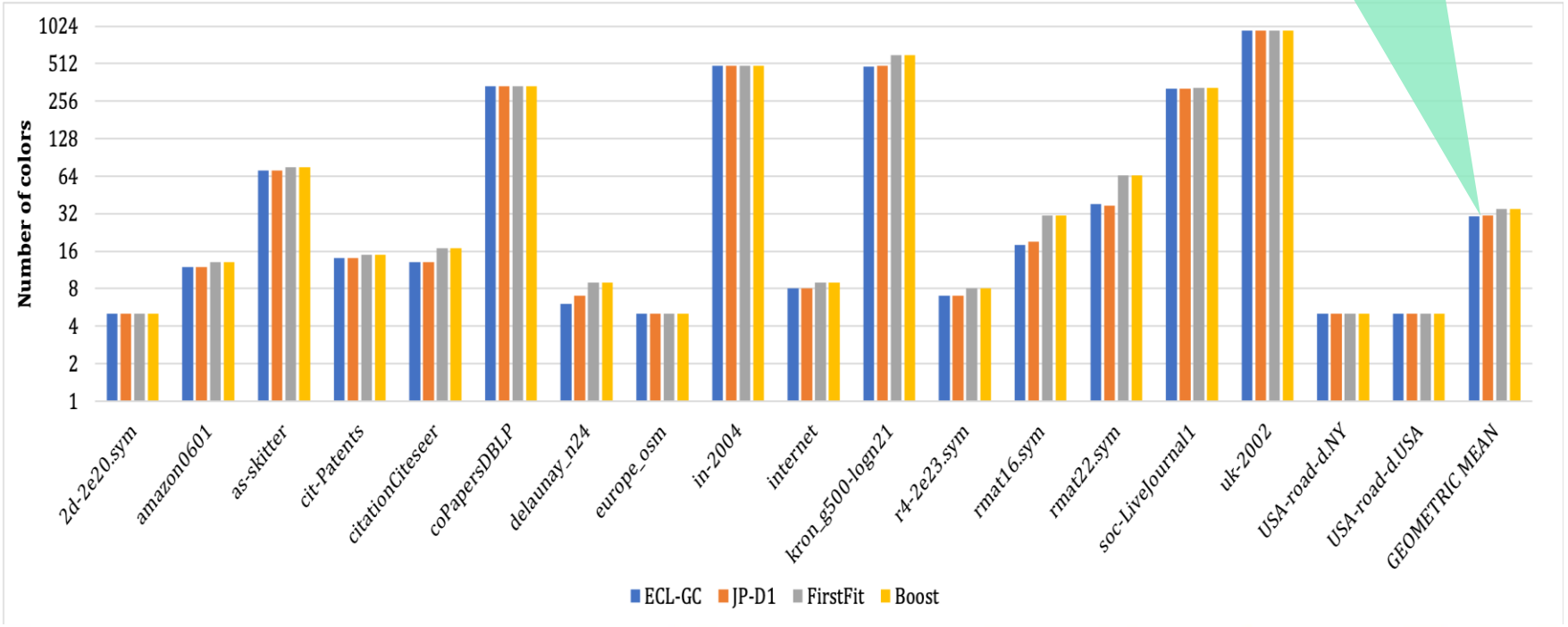
ECL-GC assigns only 1 more color on the remaining 7 graphs



Number of Colors (Serial CPU Codes)

ECL-GC assigns the smallest or the same number of colors on all tested inputs

ECL-GC's and JP-D1's coloring quality is almost identical

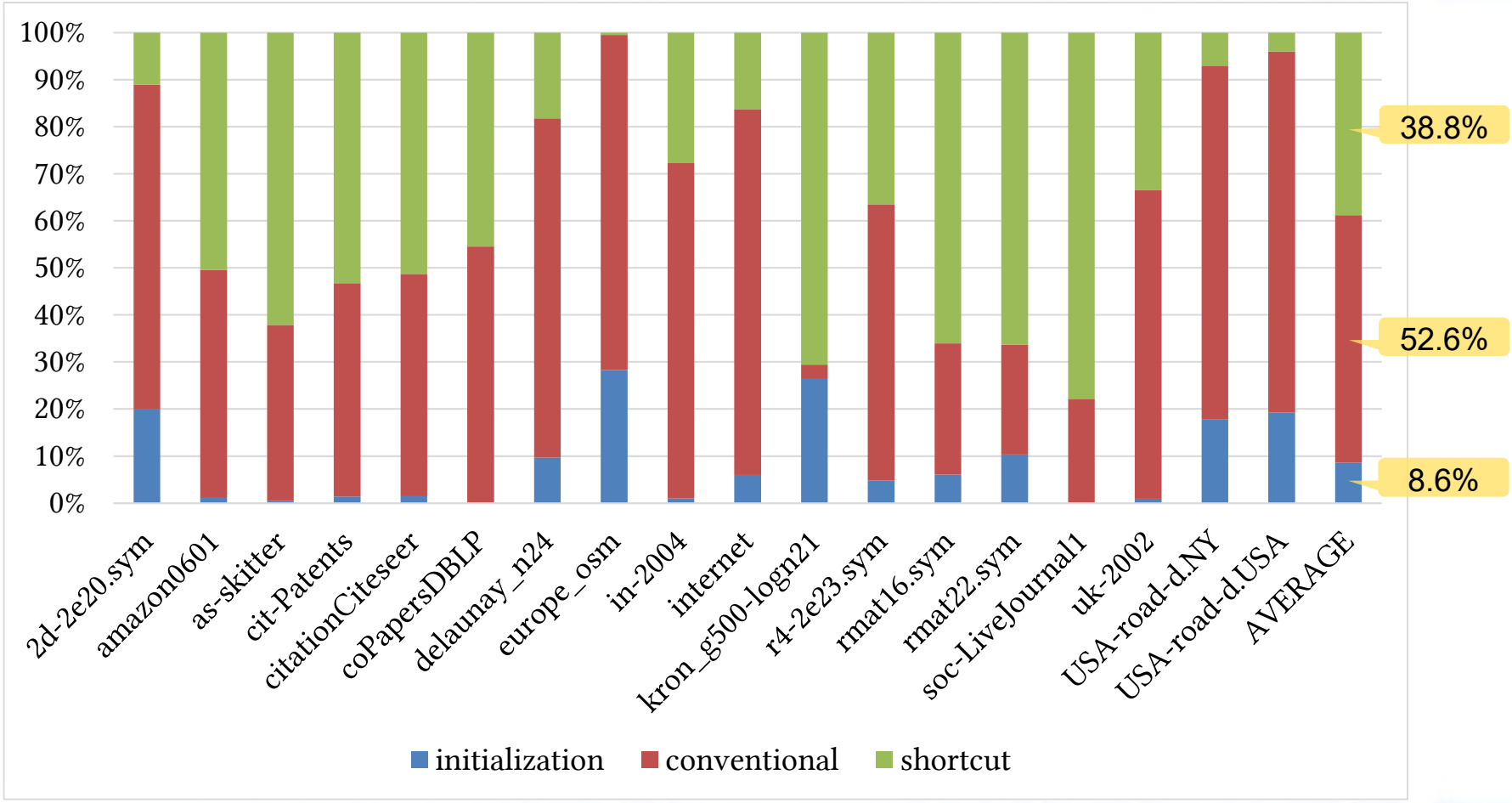


Titan V Speedup due to Shortcuts

input	baseline	+SC1	+SC2	+SC1+SC2
2d-2e20.sym	1.000	1.046	1.005	1.092
amazon0601	1.000	1.236	1.075	1.285
as-skitter	1.000	3.957	1.001	4.198
citationCiteseer	1.000	1.751	1.057	1.816
cit-Patents	1.000	2.015	1.258	2.123
coPapersDBLP	1.000	4.410	1.004	4.407
delaunay_n24	1.000	1.126	1.037	1.168
europe_osm	1.000	1.025	0.999	1.028
in-2004	1.000	1.051	1.019	1.030
internet	1.000	1.248	1.016	1.284
kron_g500-logn21	1.000	70.378	1.004	70.179
r4-2e23.sym	1.000	1.250	1.110	1.339
rmat16.sym	1.000	5.112	1.008	5.251
rmat22.sym	1.000	9.958	0.989	10.163
soc-LiveJournal1	1.000	16.026	0.996	16.028
uk-2002	1.000	2.590	1.010	2.612
USA-road-d.NY	1.000	1.000	1.014	1.027
USA-road-d.USA	1.000	1.068	1.003	1.073
geo mean	1.000	2.570	1.032	2.632

Shortcut 1 is much more important than Shortcut 2

Source of Color Assignment



Summary and Conclusion

- New algorithmic optimizations for graph coloring
  - Increases parallelism by breaking data dependencies
    - Exceeds intrinsic (LDF) DAG parallelism by 2.5x
  - These "shortcuts" guarantee same final coloring
- ECL-GC GPU performance
  - Faster on average than prior CPU and GPU codes
    - Especially on high-degree graphs
- Conclusion
  - ECL-GC can speed up apps that use graph coloring
  - Look for shortcuts to increase parallelism in other fields

Thank you!

- Acknowledgments
  - NSF, NVIDIA
- Contact information
  - burtscher@txstate.edu
- Web page (link to paper and source code)
  - http://cs.txstate.edu/~burtscher/research/ECL-GC/

ECL
Efficient Computing Laboratory

TEXAS STATE UNIVERSITY
The rising STAR of Texas

ACM Symposium on Principles and Practice of Parallel Programming (PPoPP), pp. 262-275. February 2020.